

1. Objective

Task 1 implements a fully connected neural network (FCNN) for three-class classification on two synthetic two-dimensional datasets. Dataset 1 contains compact Gaussian-like clusters designed to be linearly separable. Dataset 2 contains three interleaving spiral arms and therefore requires nonlinear decision boundaries.

The notebook implements the network and backpropagation from scratch using NumPy. Dataset 1 is trained with one hidden layer and Dataset 2 with two hidden layers. Multiple hidden-node configurations are evaluated and the best architecture is selected using validation accuracy.

2. Experimental Setup

- Random seed: 10.
- Three classes with 500 examples per class, giving 1,500 examples per dataset.
- Per-class split: 60% training, 20% validation, 20% test.
- Final split sizes: 900 training, 300 validation and 300 test examples.
- Inputs are standardized using training-set statistics only.
- Sigmoid activation is used in hidden and output layers.
- Training uses stochastic gradient descent with batch size 1.
- Squared error is used as the instantaneous loss.
- Learning rate used in the executed architecture search: 0.05.
- Gaussian models are trained for 100 epochs; spiral models for 300 epochs.

3. Dataset Generation

Three-class Gaussian clusters

Three two-dimensional multivariate Gaussian classes are generated with 500 points per class. The selected means and covariance matrices in the notebook create compact, well-separated clusters.

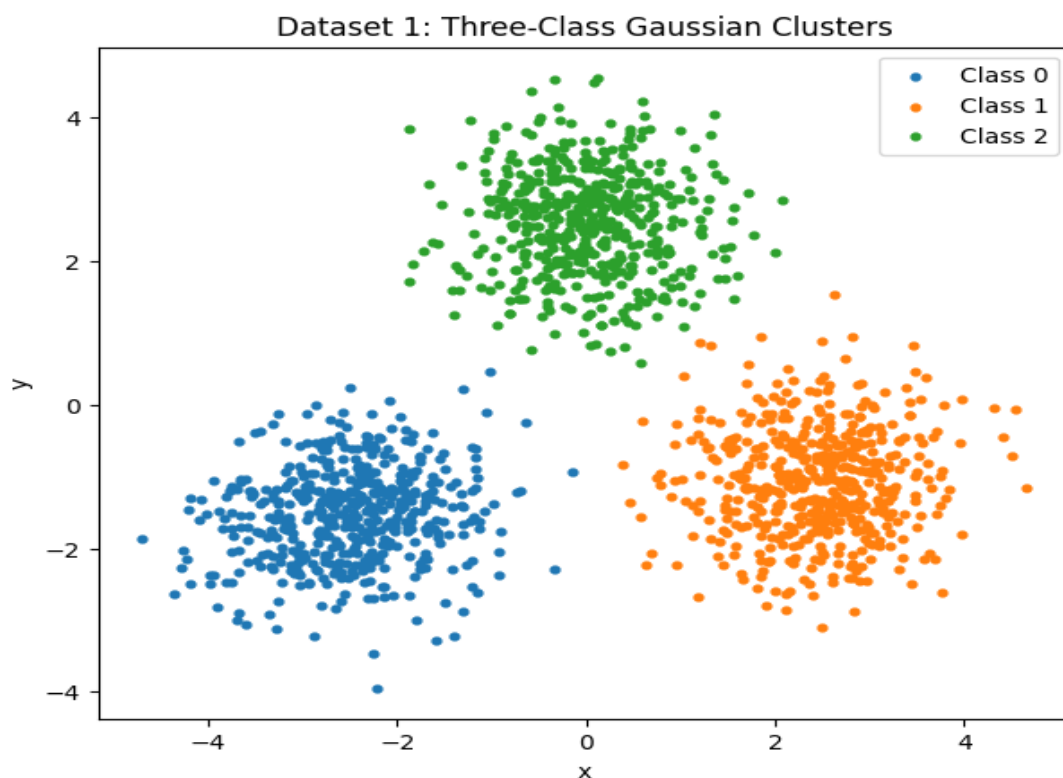


Figure 1. Generated Gaussian dataset.

Three interleaving spiral arms are generated parametrically using radius and angle, with random angular and radial perturbations. This produces a nonlinear classification problem.

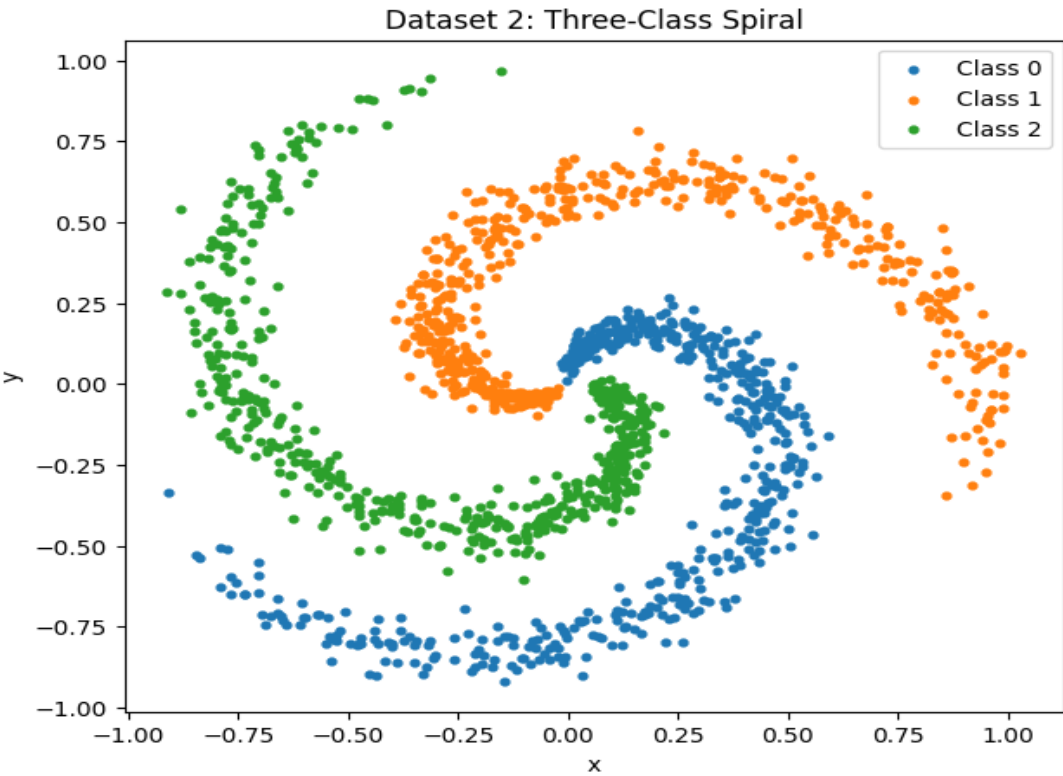


Figure 2. Generated spiral dataset.

4. Data Splitting and Preprocessing

Each class is shuffled independently before splitting. For each of the three classes, 300 samples are used for training, 100 for validation and 100 for testing. The resulting datasets therefore contain 900 training, 300 validation and 300 test samples.

Standardization is fitted on the training set and the same training mean and standard deviation are then applied to validation and test data. This avoids using information from the held-out sets during preprocessing.

5. FCNN Architecture

Dataset 1 uses the form **2 -> H -> 3**, where H is the number of hidden nodes. The notebook evaluates $H = 3, 5, 8, 10, 15$ and 20.

Dataset 2 uses the form **2 -> H1 -> H2 -> 3**. The notebook evaluates the hidden-layer pairs (5,5), (8,8), (10,10), (15,10), (15,15), (20,15) and (20,20).

6. Forward Propagation, Loss and Backpropagation

For each layer, the pre-activation value is computed and passed through the sigmoid activation. The notebook defines the sigmoid and its derivative as follows:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(a) = a(1 - a)$$

For one training example, the instantaneous squared error is:

$$L = \frac{1}{2} \sum_k (t_k - o_k)^2$$

The output error signal is obtained from the squared-error derivative multiplied by the sigmoid derivative, and the error is then propagated backwards through the hidden layers.

The parameter update used after each training example is:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

Because the weights and biases are updated immediately after every individual example, the training procedure is stochastic gradient descent with batch size 1.

7. Architecture Search - Gaussian Dataset

Architecture	Train Accuracy	Validation Accuracy	Validation Confusion Matrix
(2, 3, 3)	100.00%	99.67%	[99 1 0] [0 100 0] [0 0 100]
(2, 5, 3)	100.00%	99.67%	[99 1 0] [0 100 0] [0 0 100]
(2, 8, 3)	99.89%	100.00%	[100 0 0] [0 100 0] [0 0 100]
(2, 10, 3)	100.00%	100.00%	[100 0 0] [0 100 0] [0 0 100]
(2, 15, 3)	99.89%	100.00%	[100 0 0] [0 100 0] [0 0 100]
(2, 20, 3)	99.89%	100.00%	[100 0 0] [0 100 0] [0 0 100]

The validation sweep shows that the (2, 8, 3) architecture is the first architecture in the tested order to achieve 100% validation accuracy, so it is selected as the best Gaussian architecture by the notebook's max-validation rule.

8. Architecture Search - Spiral Dataset

Architecture	Train Accuracy	Validation Accuracy	Validation Confusion Matrix
(2, 5, 5, 3)	99.11%	98.67%	[96 4 0] [0 100 0] [0 0 100]
(2, 8, 8, 3)	99.00%	97.67%	[96 3 1] [0 100 0] [1 2 97]
(2, 10, 10, 3)	99.78%	100.00%	[100 0 0] [0 100 0] [0 0 100]
(2, 15, 10, 3)	99.89%	100.00%	[100 0 0] [0 100 0] [0 0 100]
(2, 15, 15, 3)	99.78%	100.00%	[100 0 0] [0 100 0] [0 0 100]
(2, 20, 15, 3)	99.89%	100.00%	[100 0 0] [0 100 0] [0 0 100]
(2, 20, 20, 3)	99.89%	99.67%	[99 1 0] [0 100 0] [0 0 100]

The (2, 10, 10, 3) architecture is selected because it is the first architecture in the tested order to achieve the maximum validation accuracy of 100%.

9. Average Error versus Epochs

The assignment asks for the average error versus epochs for the best architecture. The notebook's training histories are shown below.

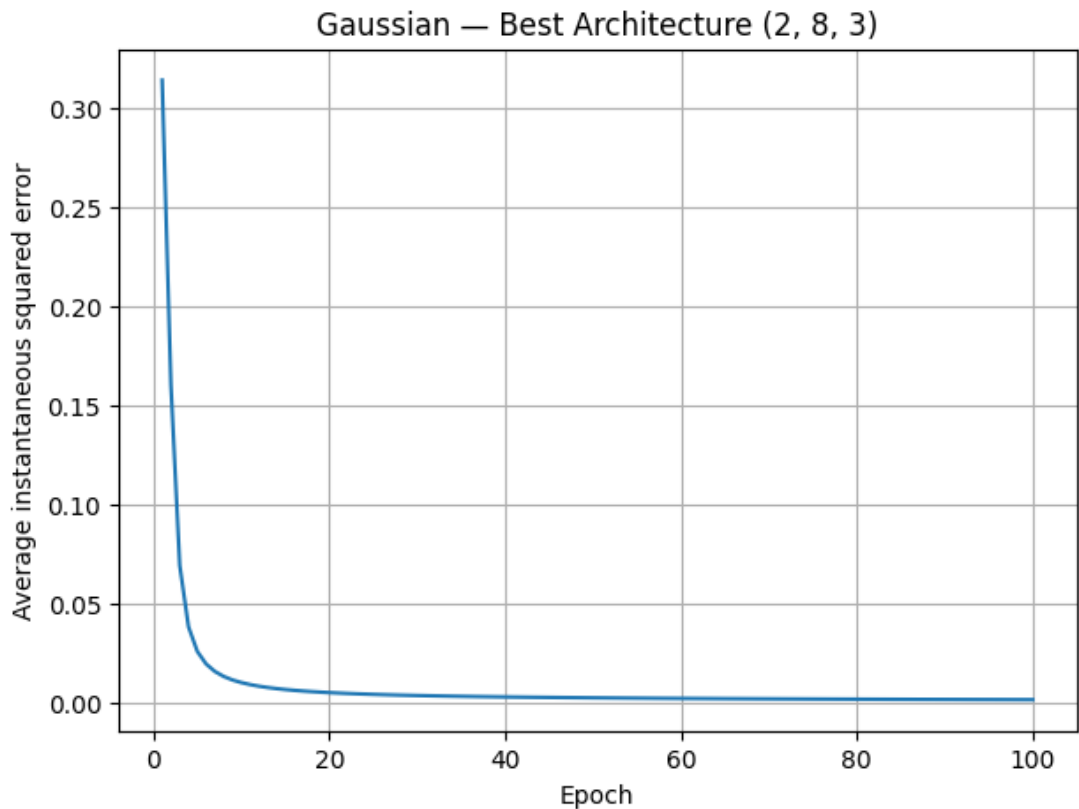


Figure 3. Gaussian dataset - best architecture (2, 8, 3).

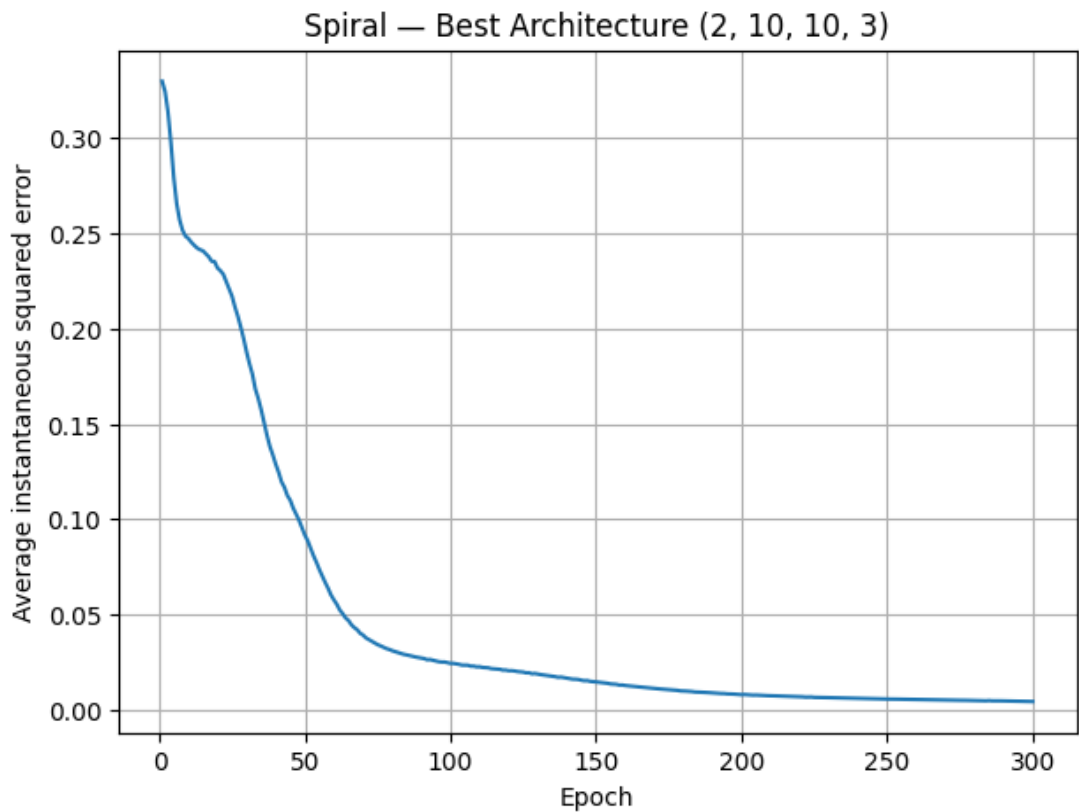


Figure 4. Spiral dataset - best architecture (2, 10, 10, 3).

10. Final Validation and Test Results

Gaussian dataset

Selected architecture: (2, 8, 3)

Validation accuracy: 100.00%

Test accuracy: 100.00%

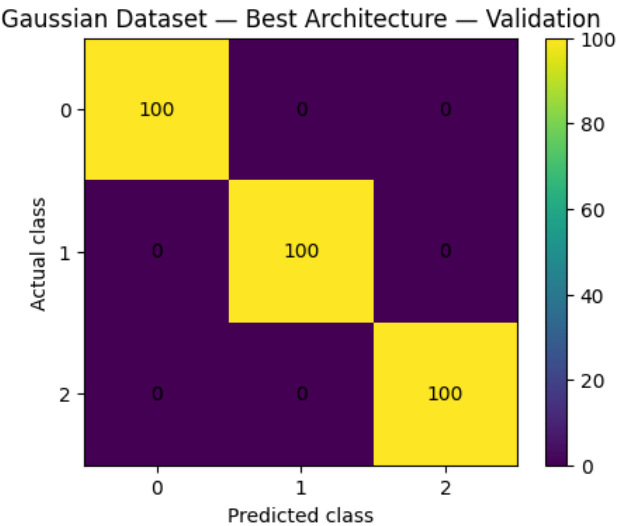


Figure 5. Gaussian validation confusion matrix.

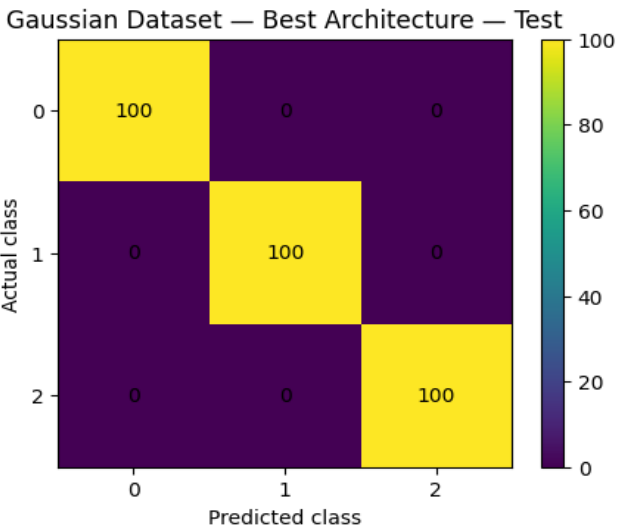


Figure 6. Gaussian test confusion matrix.

11. Final Validation and Test Results - Spiral Dataset

Selected architecture: (2, 10, 10, 3)

Validation accuracy: 100.00%

Test accuracy: 100.00%

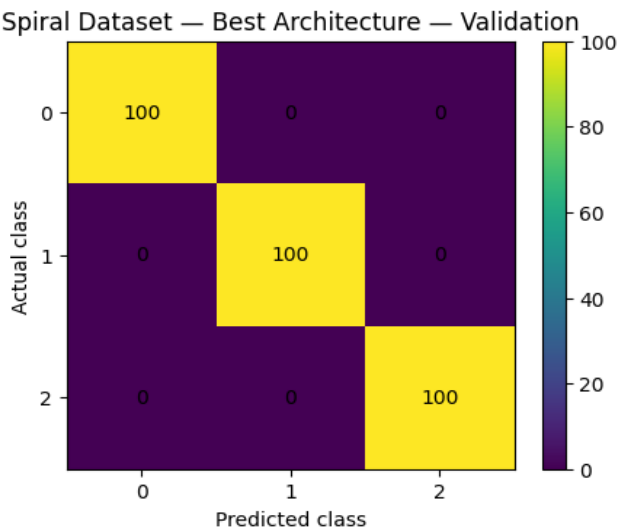


Figure 7. Spiral validation confusion matrix.

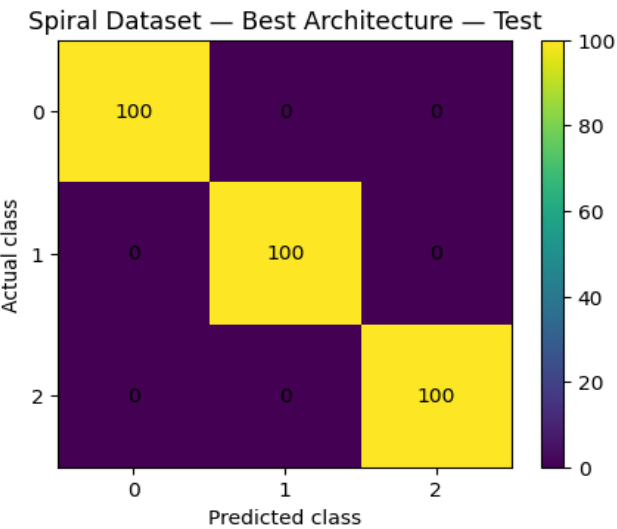


Figure 8. Spiral test confusion matrix.

12. Decision Region Plots

The best architecture for each dataset is evaluated over a dense two-dimensional grid, and the predicted class regions are superimposed with the training data.

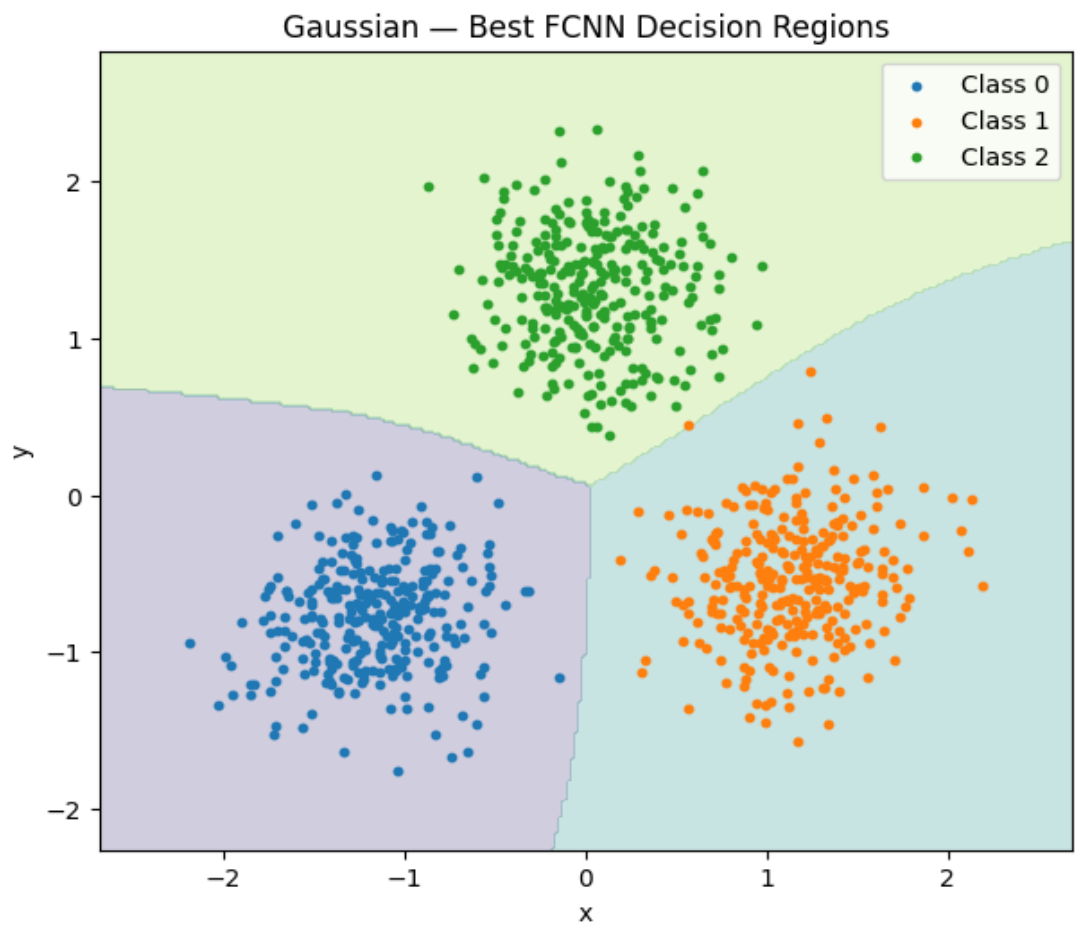


Figure 9. Gaussian decision regions with training data.

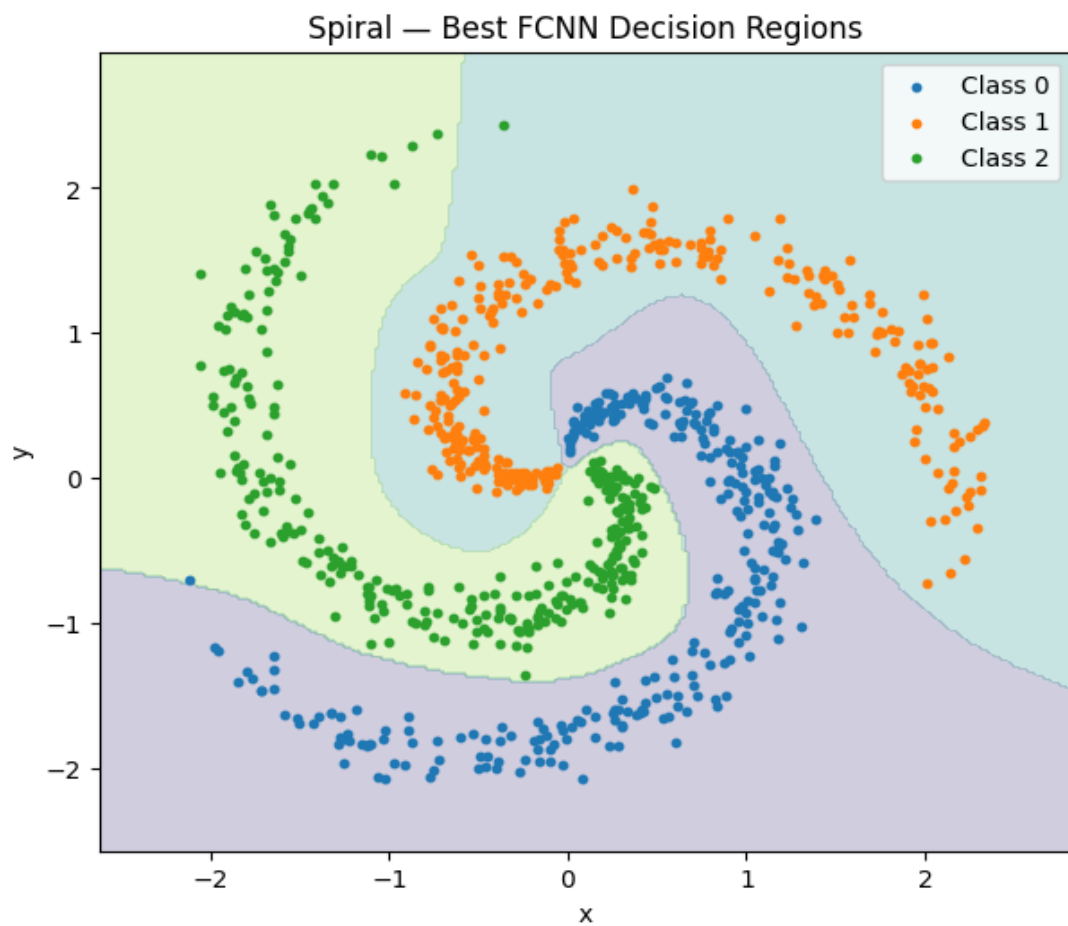
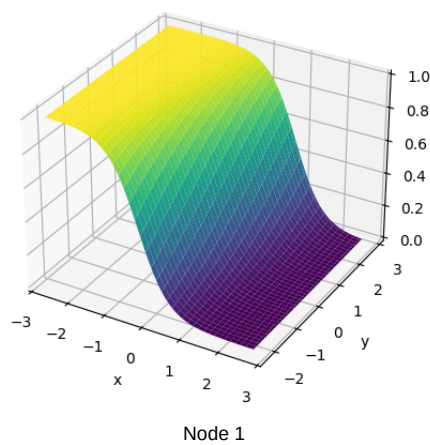


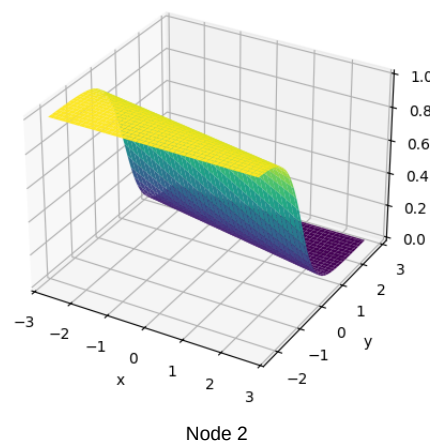
Figure 10. Spiral decision regions with training data.

Gaussian - Training - Hidden layer 1

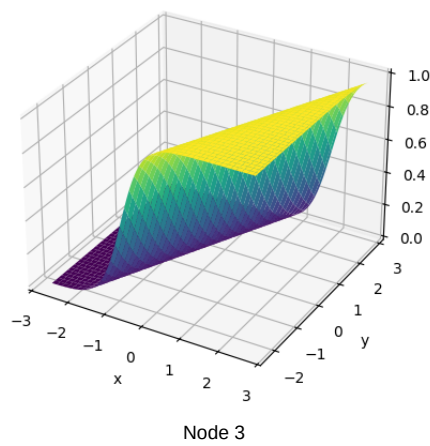
Gaussian — X_train | Layer 1, Node 1



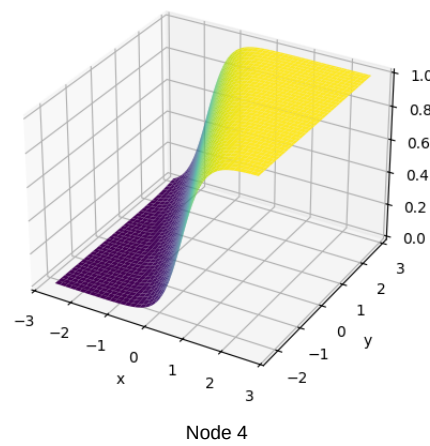
Gaussian — X_train | Layer 1, Node 2



Gaussian — X_train | Layer 1, Node 3

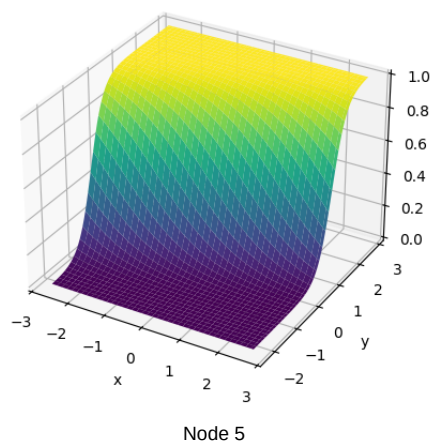


Gaussian — X_train | Layer 1, Node 4

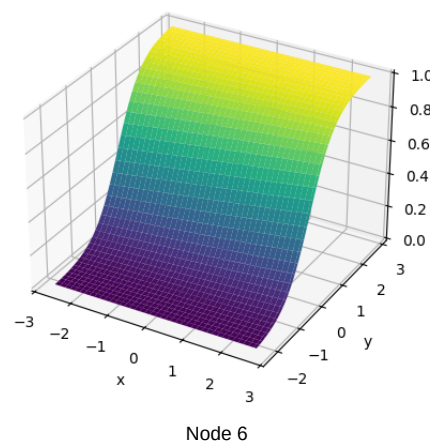


Gaussian - Training - Hidden layer 1 (continued)

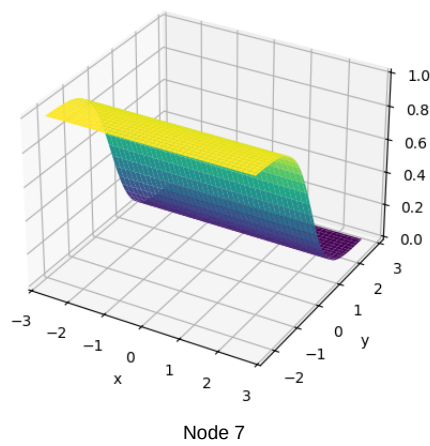
Gaussian — X_train | Layer 1, Node 5



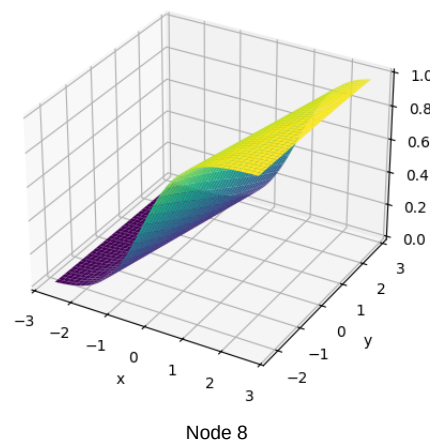
Gaussian — X_train | Layer 1, Node 6



Gaussian — X_train | Layer 1, Node 7

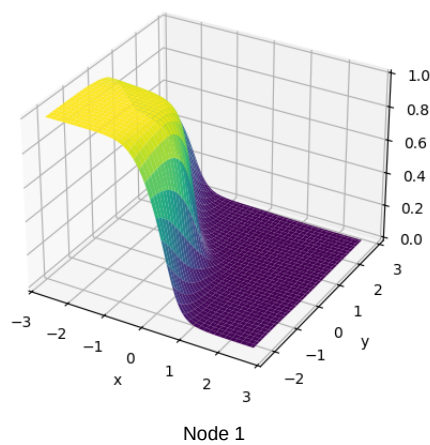


Gaussian — X_train | Layer 1, Node 8

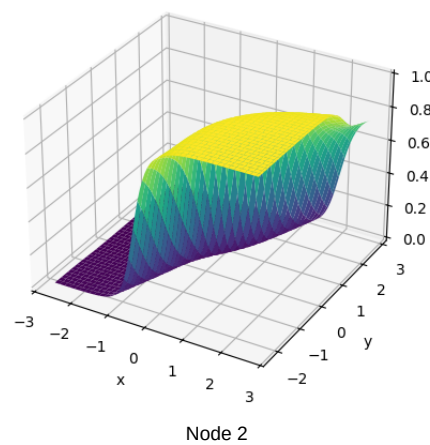


Gaussian - Training - Output layer

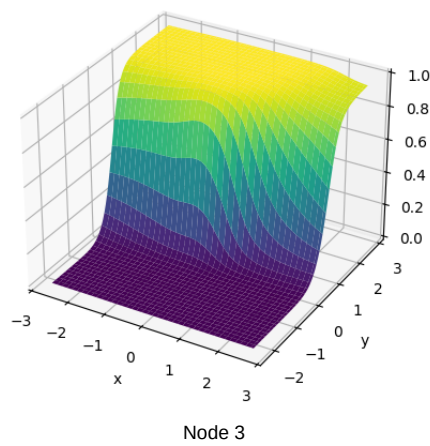
Gaussian — X_train | Layer 2, Node 1



Gaussian — X_train | Layer 2, Node 2

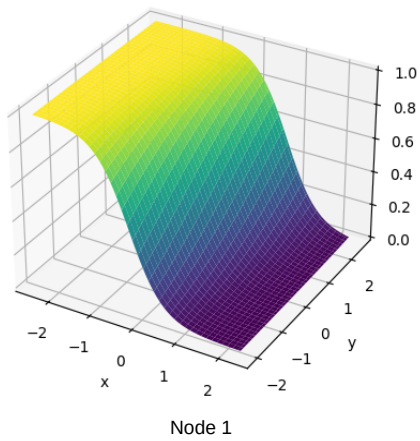


Gaussian — X_train | Layer 2, Node 3

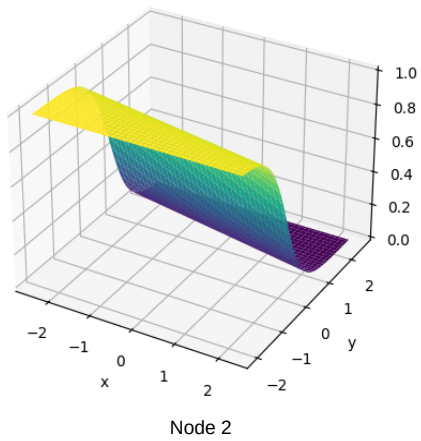


Gaussian - Validation - Hidden layer 1

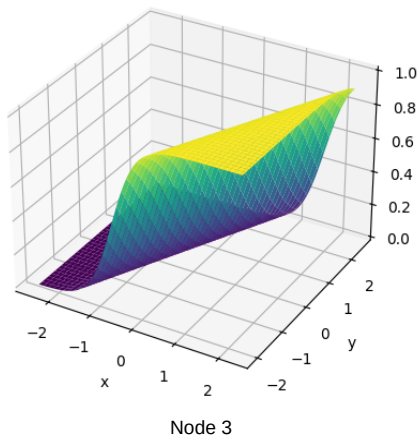
Gaussian — X_val | Layer 1, Node 1



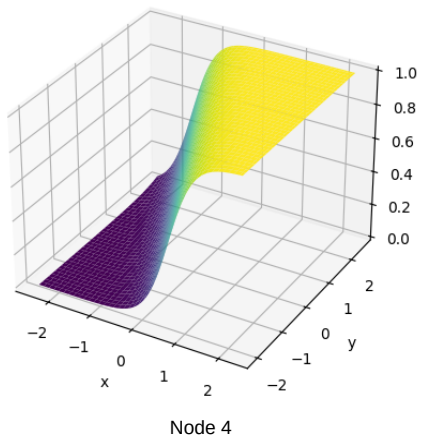
Gaussian — X_val | Layer 1, Node 2



Gaussian — X_val | Layer 1, Node 3

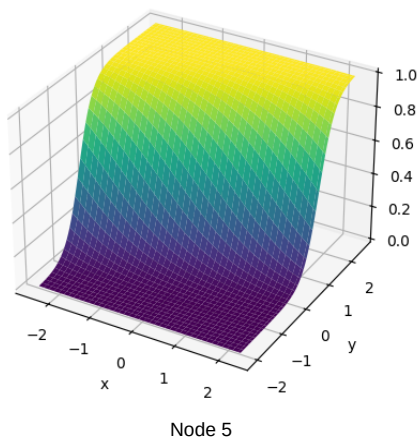


Gaussian — X_val | Layer 1, Node 4

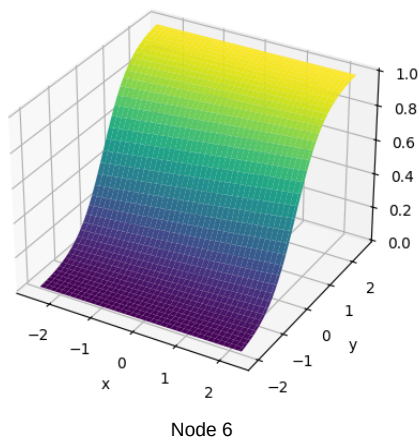


Gaussian - Validation - Hidden layer 1 (continued)

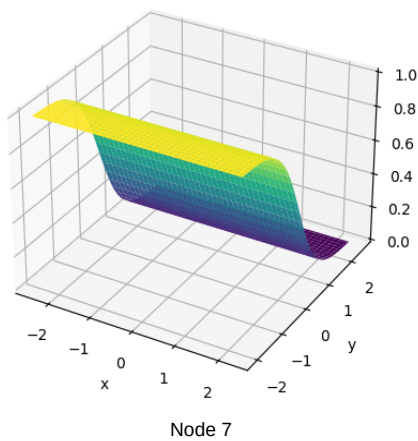
Gaussian — X_val | Layer 1, Node 5



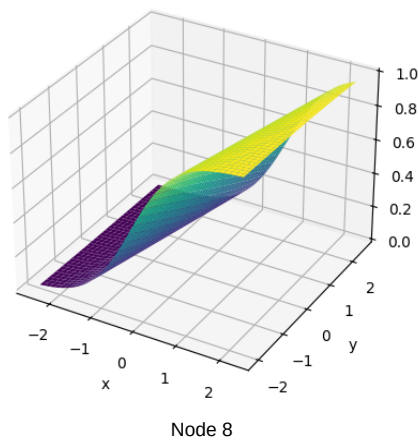
Gaussian — X_val | Layer 1, Node 6



Gaussian — X_val | Layer 1, Node 7

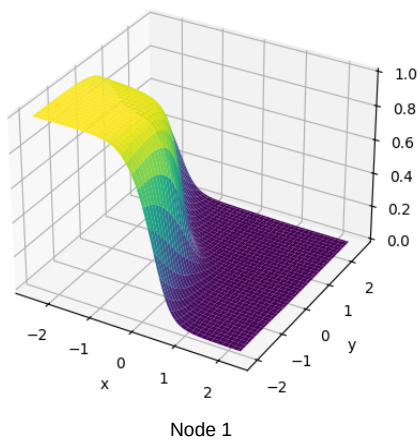


Gaussian — X_val | Layer 1, Node 8

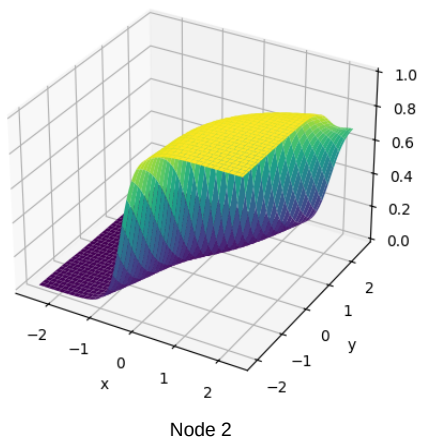


Gaussian - Validation - Output layer

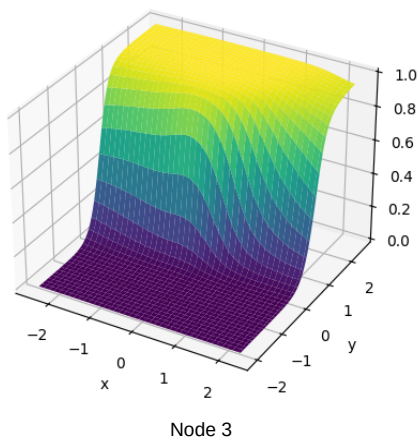
Gaussian — X_val | Layer 2, Node 1



Gaussian — X_val | Layer 2, Node 2

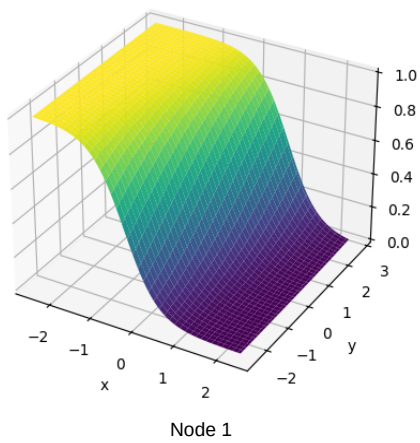


Gaussian — X_val | Layer 2, Node 3

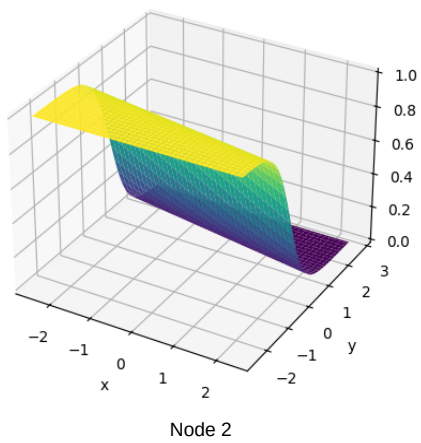


Gaussian - Test - Hidden layer 1

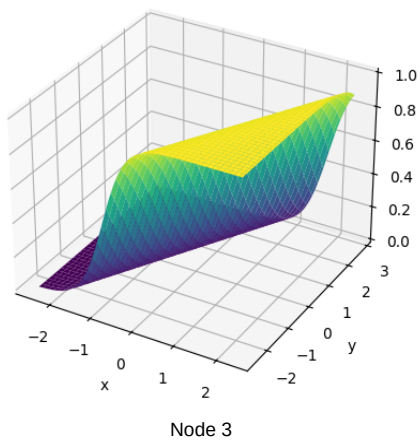
Gaussian — X_test | Layer 1, Node 1



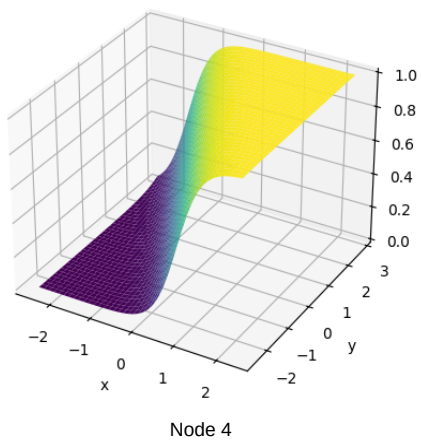
Gaussian — X_test | Layer 1, Node 2



Gaussian — X_test | Layer 1, Node 3

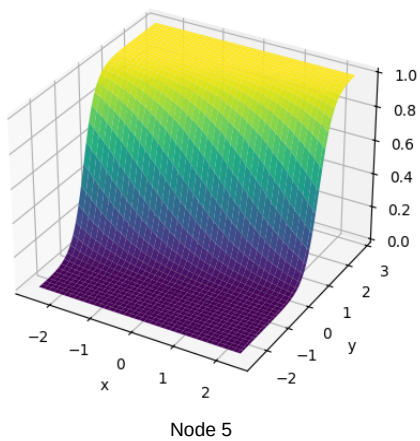


Gaussian — X_test | Layer 1, Node 4

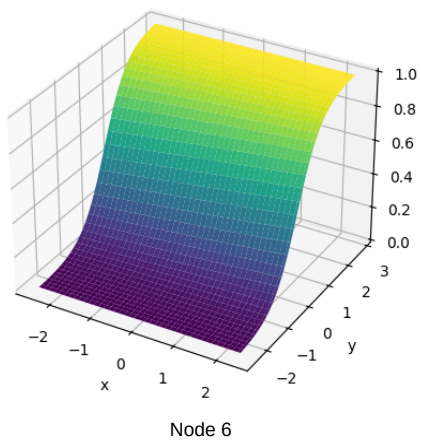


Gaussian - Test - Hidden layer 1 (continued)

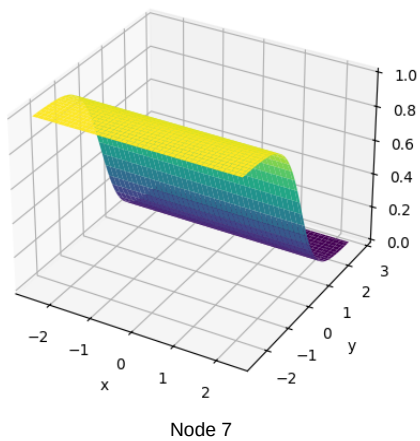
Gaussian — X_test | Layer 1, Node 5



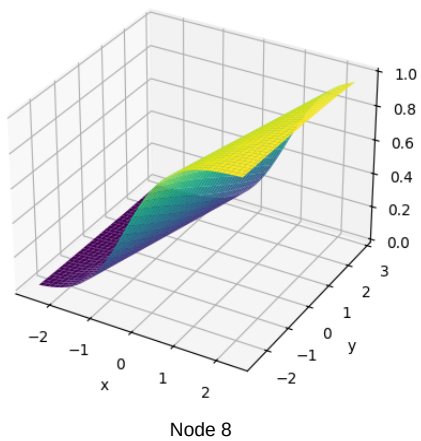
Gaussian — X_test | Layer 1, Node 6



Gaussian — X_test | Layer 1, Node 7

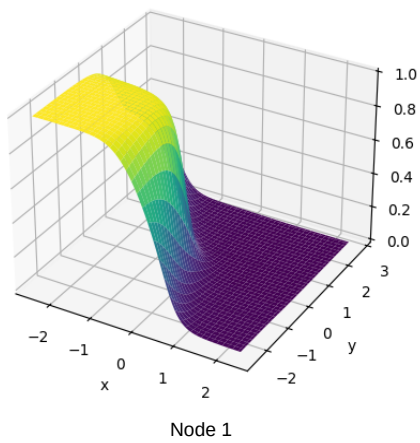


Gaussian — X_test | Layer 1, Node 8

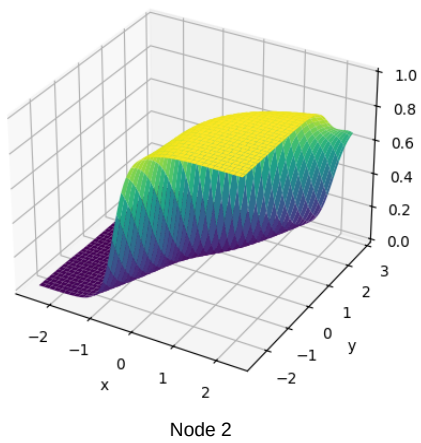


Gaussian - Test - Output layer

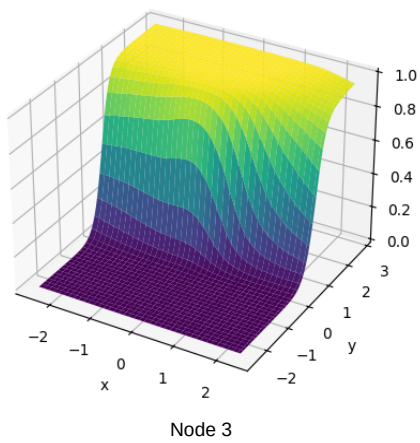
Gaussian — X_test | Layer 2, Node 1



Gaussian — X_test | Layer 2, Node 2

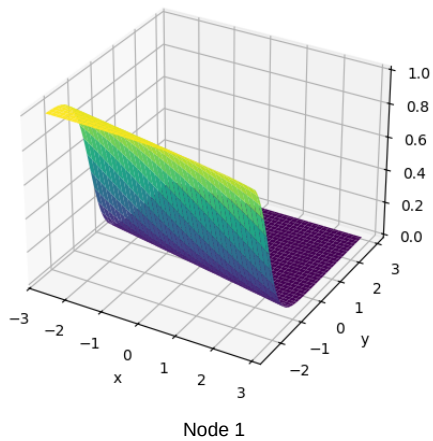


Gaussian — X_test | Layer 2, Node 3

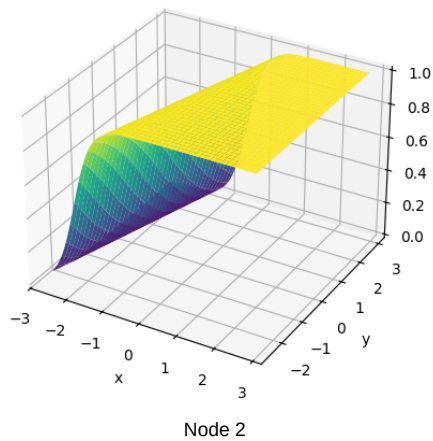


Spiral - Training - Hidden layer 1

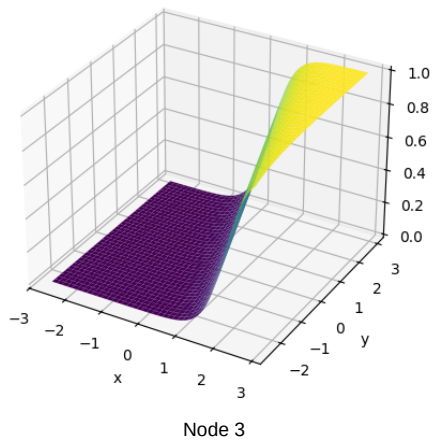
Spiral — X_train | Layer 1, Node 1



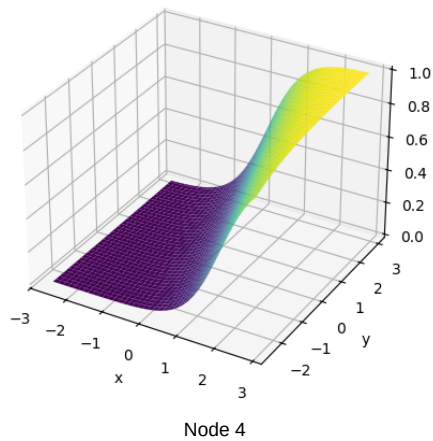
Spiral — X_train | Layer 1, Node 2



Spiral — X_train | Layer 1, Node 3

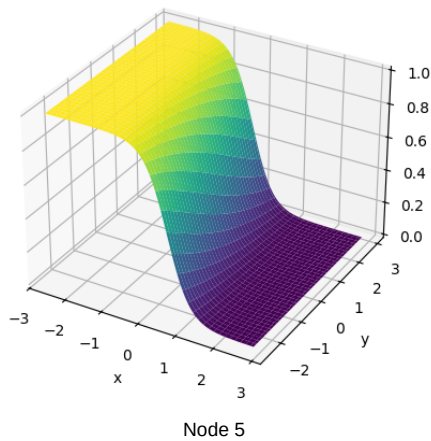


Spiral — X_train | Layer 1, Node 4

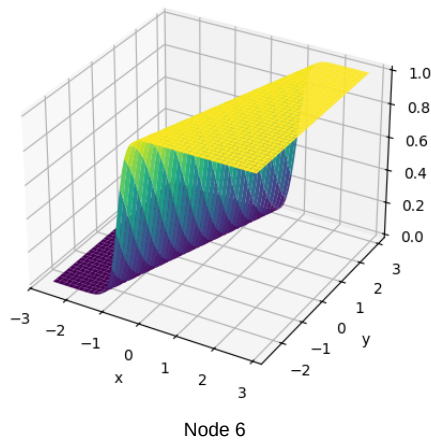


Spiral - Training - Hidden layer 1 (continued)

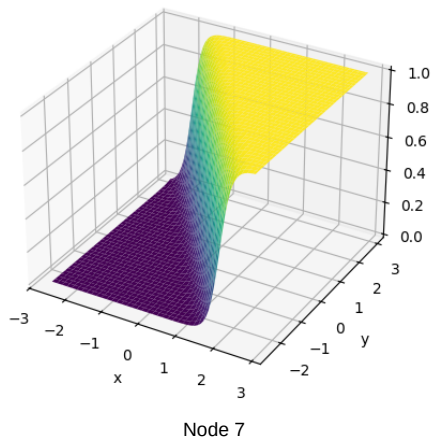
Spiral — X_train | Layer 1, Node 5



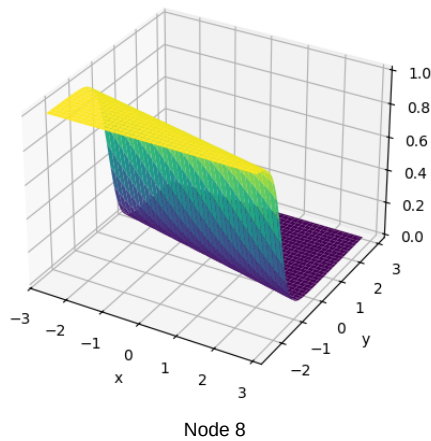
Spiral — X_train | Layer 1, Node 6



Spiral — X_train | Layer 1, Node 7

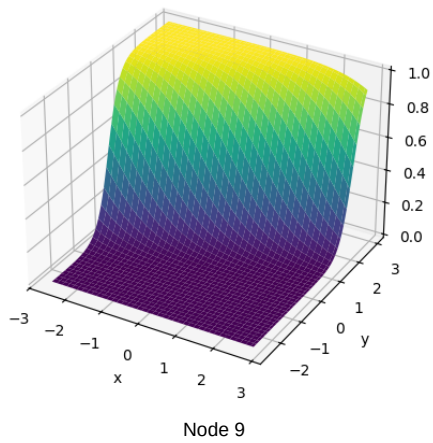


Spiral — X_train | Layer 1, Node 8

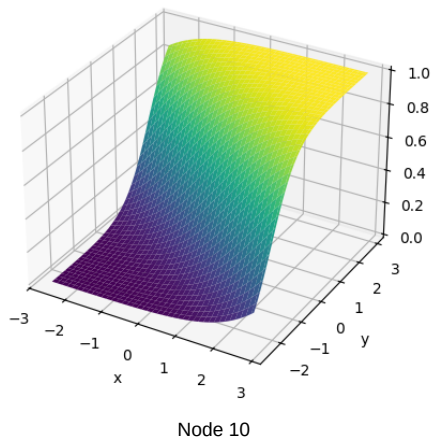


Spiral - Training - Hidden layer 1 (continued)

Spiral — X_train | Layer 1, Node 9

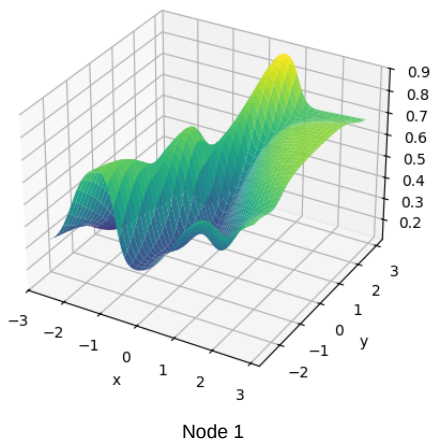


Spiral — X_train | Layer 1, Node 10

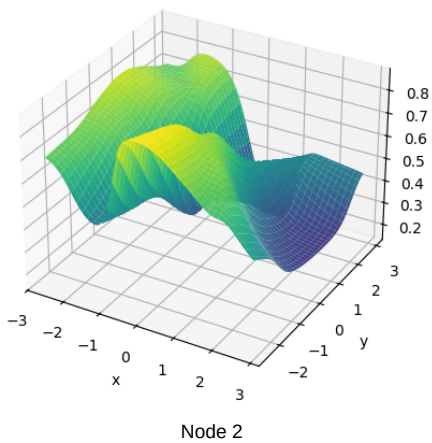


Spiral - Training - Hidden layer 2

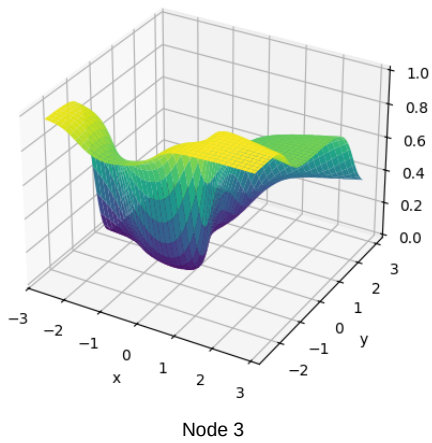
Spiral — X_train | Layer 2, Node 1



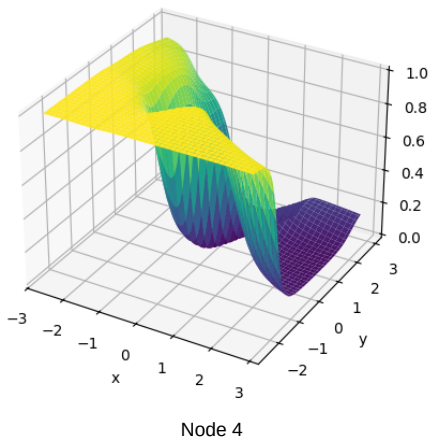
Spiral — X_train | Layer 2, Node 2



Spiral — X_train | Layer 2, Node 3

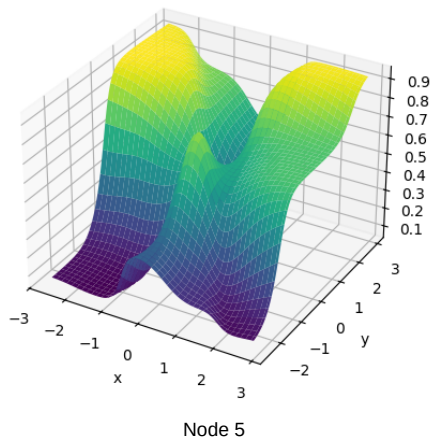


Spiral — X_train | Layer 2, Node 4

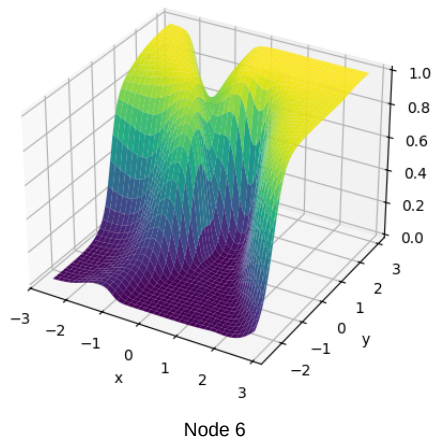


Spiral - Training - Hidden layer 2 (continued)

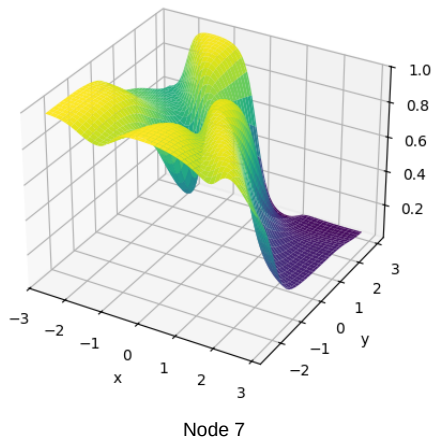
Spiral — X_train | Layer 2, Node 5



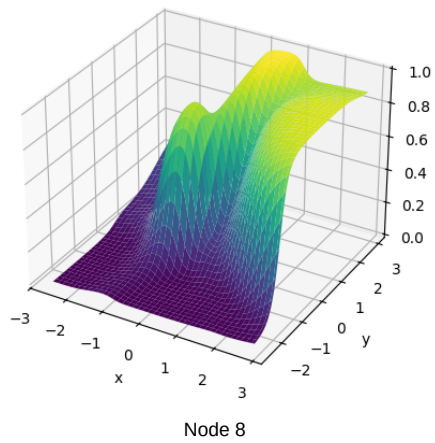
Spiral — X_train | Layer 2, Node 6



Spiral — X_train | Layer 2, Node 7

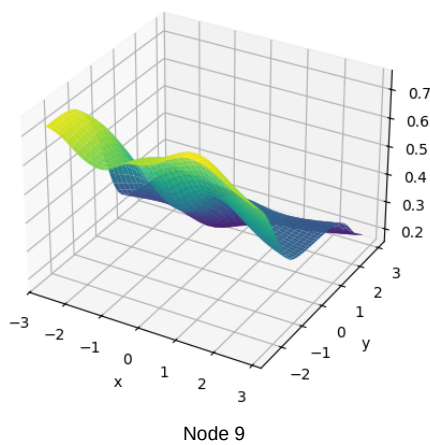


Spiral — X_train | Layer 2, Node 8

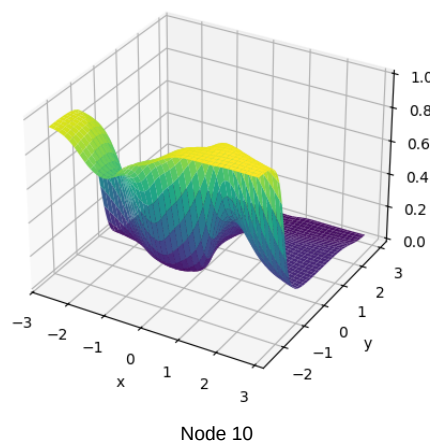


Spiral - Training - Hidden layer 2 (continued)

Spiral — X_train | Layer 2, Node 9

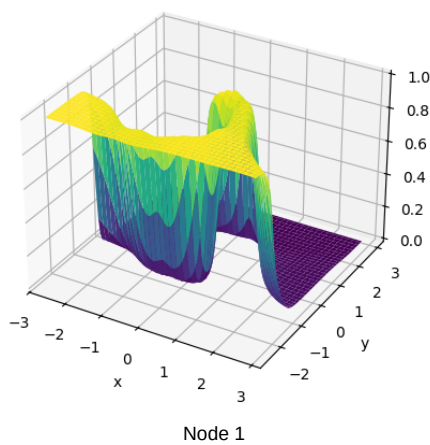


Spiral — X_train | Layer 2, Node 10

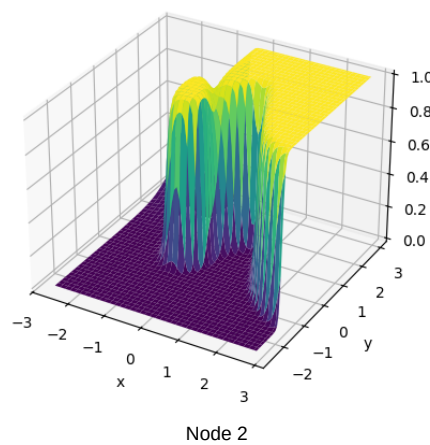


Spiral - Training - Output layer

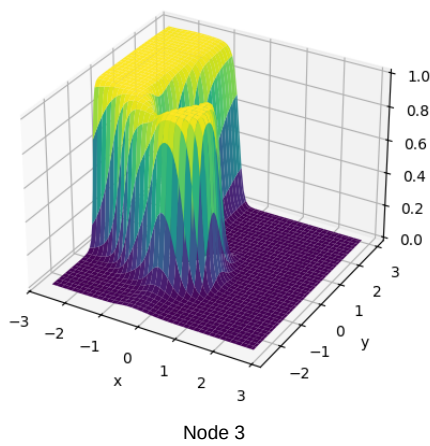
Spiral — X_train | Layer 3, Node 1



Spiral — X_train | Layer 3, Node 2

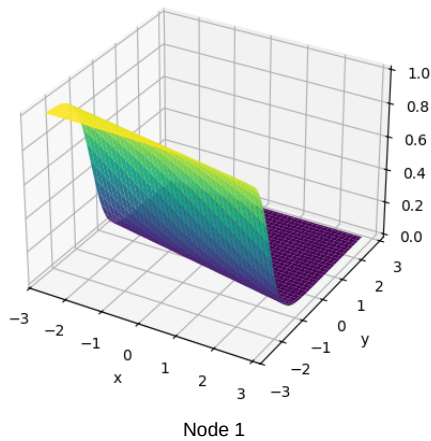


Spiral — X_train | Layer 3, Node 3

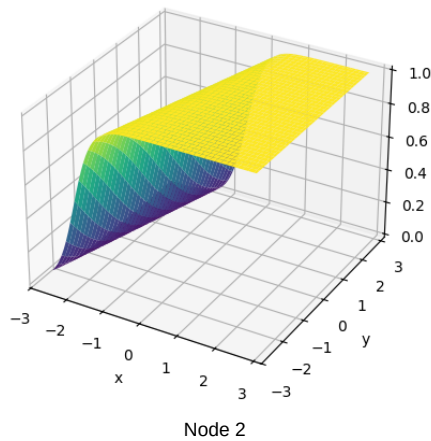


Spiral - Validation - Hidden layer 1

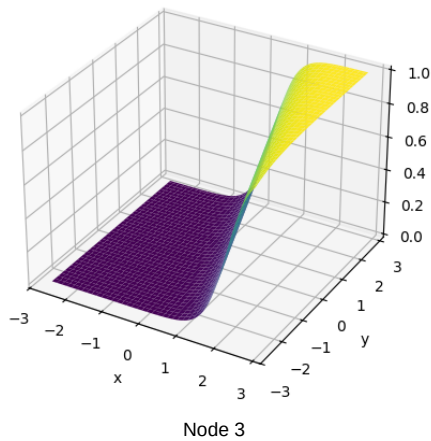
Spiral — X_val | Layer 1, Node 1



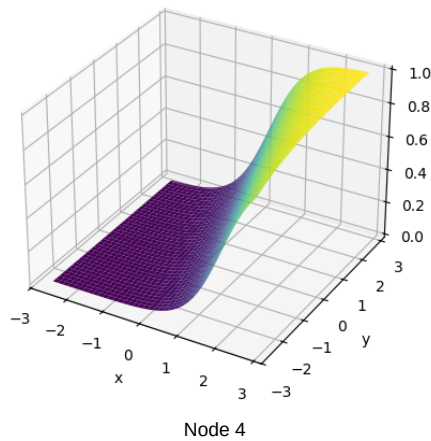
Spiral — X_val | Layer 1, Node 2



Spiral — X_val | Layer 1, Node 3

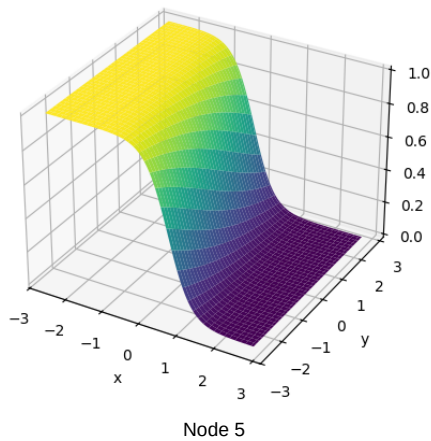


Spiral — X_val | Layer 1, Node 4

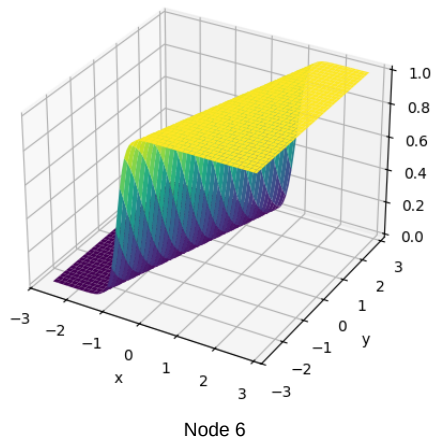


Spiral - Validation - Hidden layer 1 (continued)

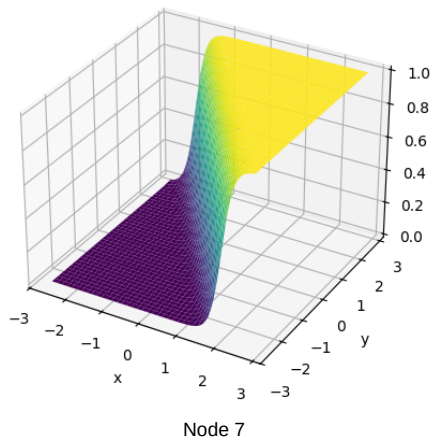
Spiral — X_val | Layer 1, Node 5



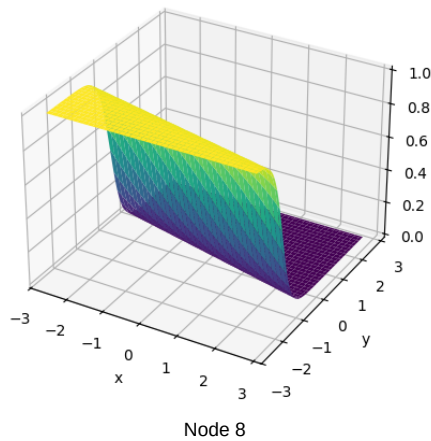
Spiral — X_val | Layer 1, Node 6



Spiral — X_val | Layer 1, Node 7

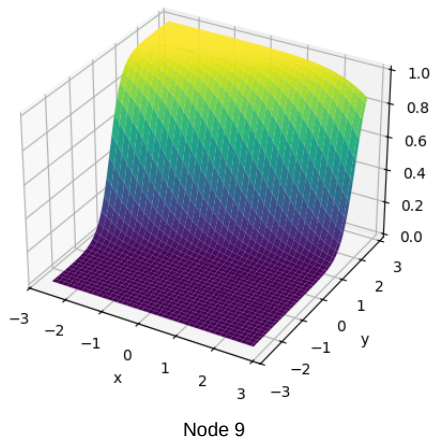


Spiral — X_val | Layer 1, Node 8

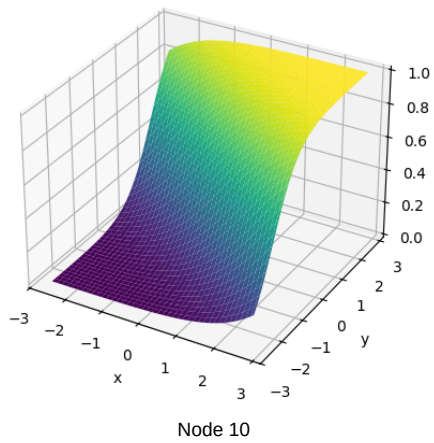


Spiral - Validation - Hidden layer 1 (continued)

Spiral — X_val | Layer 1, Node 9

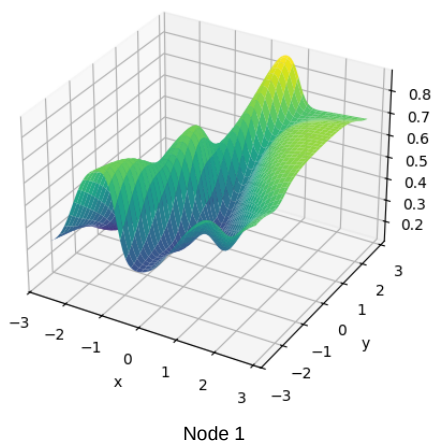


Spiral — X_val | Layer 1, Node 10

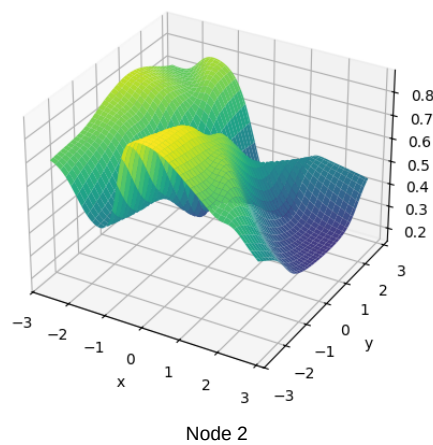


Spiral - Validation - Hidden layer 2

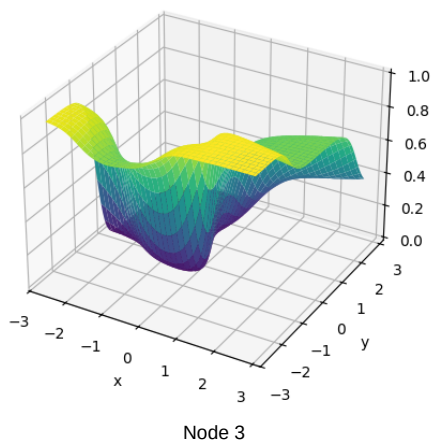
Spiral — X_val | Layer 2, Node 1



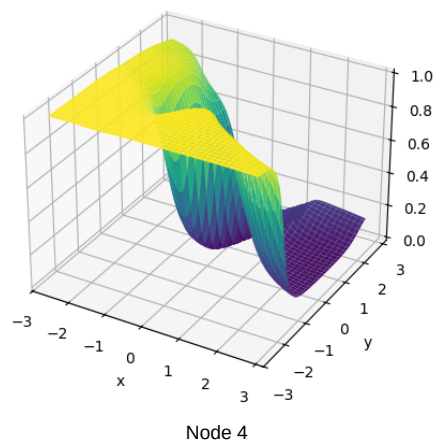
Spiral — X_val | Layer 2, Node 2



Spiral — X_val | Layer 2, Node 3

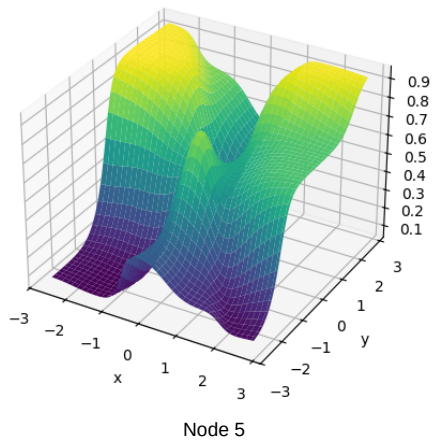


Spiral — X_val | Layer 2, Node 4

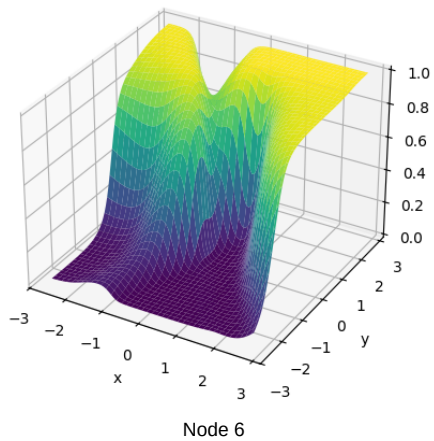


Spiral - Validation - Hidden layer 2 (continued)

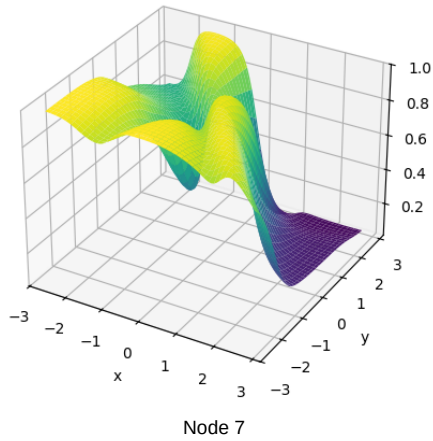
Spiral — X_val | Layer 2, Node 5



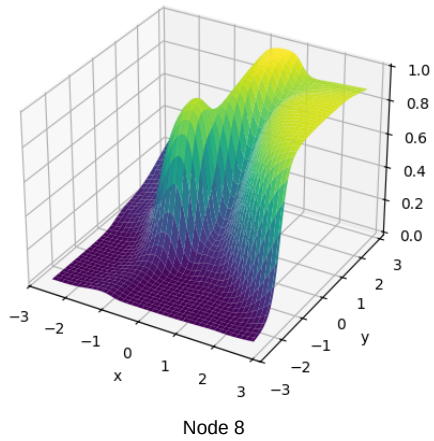
Spiral — X_val | Layer 2, Node 6



Spiral — X_val | Layer 2, Node 7

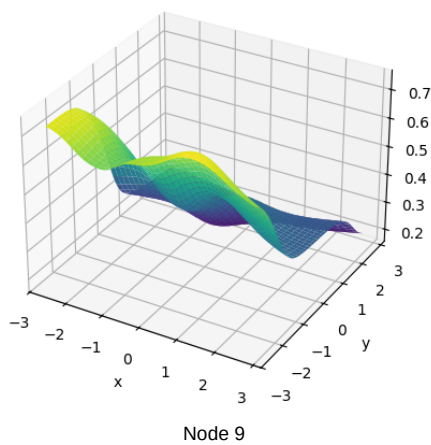


Spiral — X_val | Layer 2, Node 8

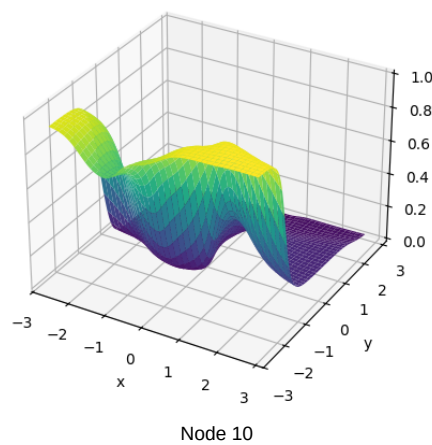


Spiral - Validation - Hidden layer 2 (continued)

Spiral — X_val | Layer 2, Node 9

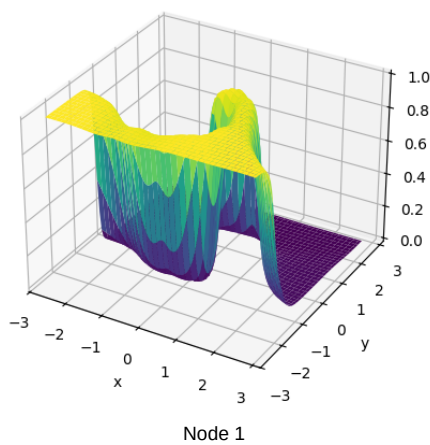


Spiral — X_val | Layer 2, Node 10

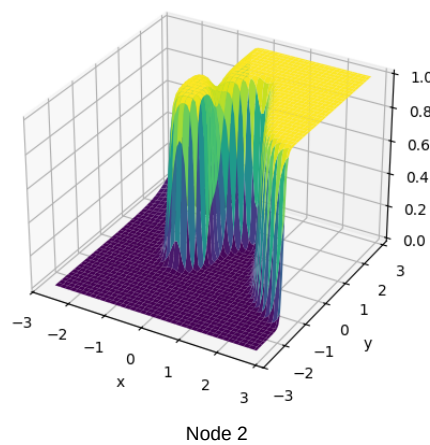


Spiral - Validation - Output layer

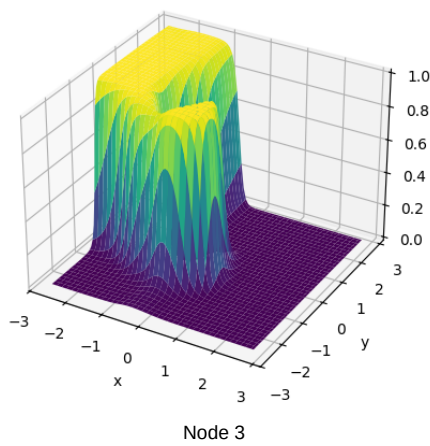
Spiral — X_val | Layer 3, Node 1



Spiral — X_val | Layer 3, Node 2

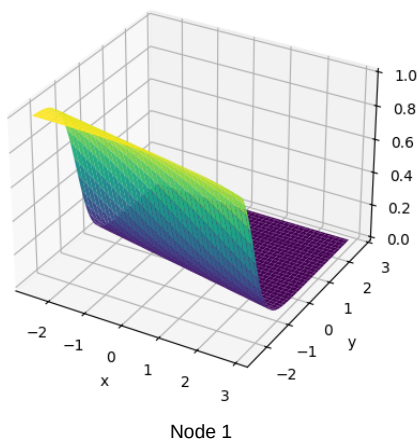


Spiral — X_val | Layer 3, Node 3

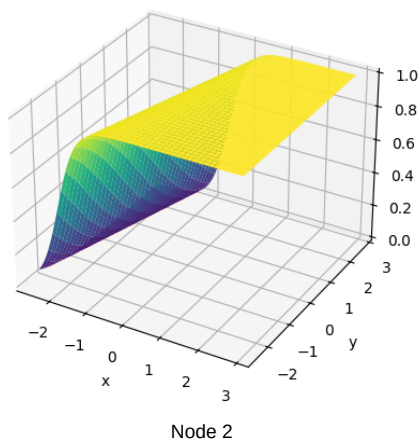


Spiral - Test - Hidden layer 1

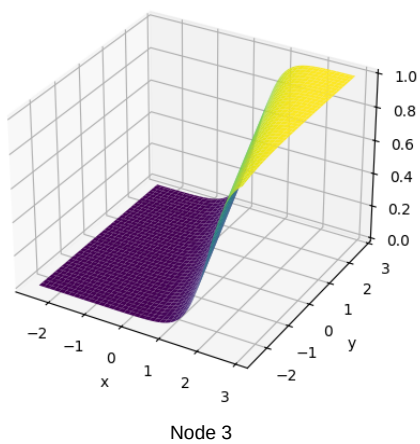
Spiral — X_test | Layer 1, Node 1



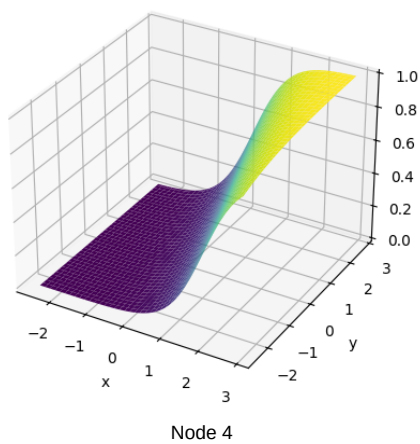
Spiral — X_test | Layer 1, Node 2



Spiral — X_test | Layer 1, Node 3

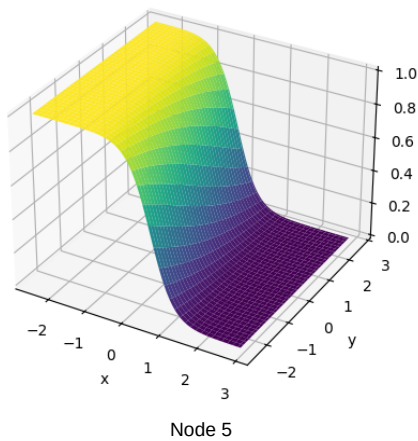


Spiral — X_test | Layer 1, Node 4

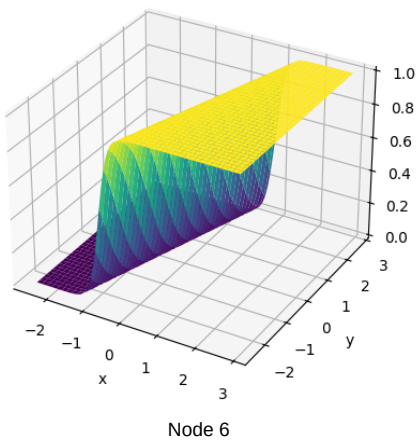


Spiral - Test - Hidden layer 1 (continued)

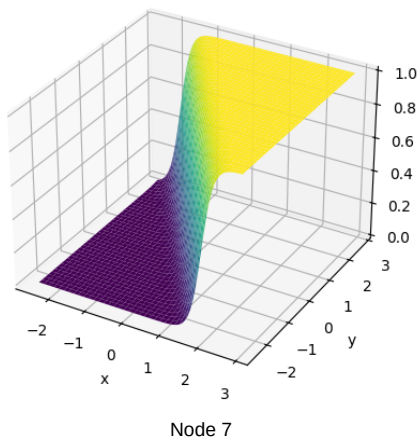
Spiral — X_test | Layer 1, Node 5



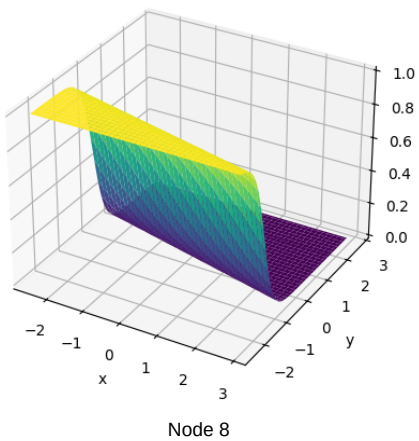
Spiral — X_test | Layer 1, Node 6



Spiral — X_test | Layer 1, Node 7

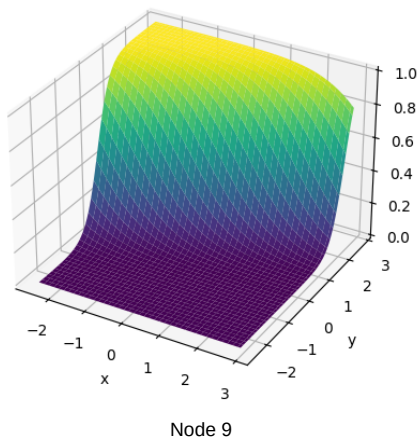


Spiral — X_test | Layer 1, Node 8

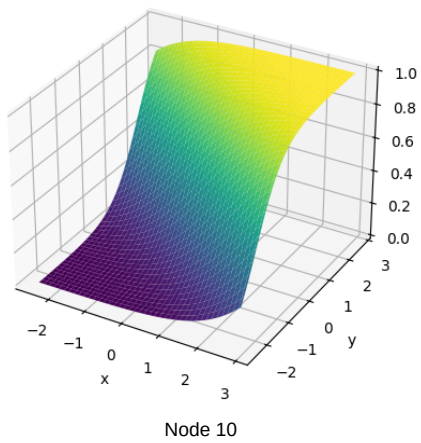


Spiral - Test - Hidden layer 1 (continued)

Spiral — X_test | Layer 1, Node 9

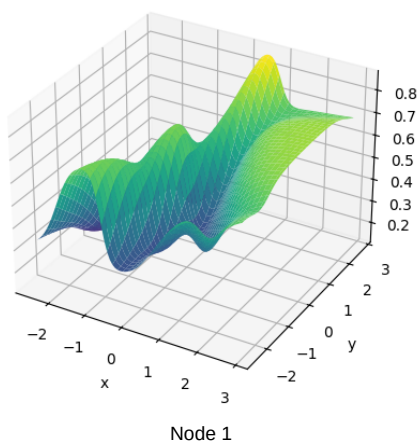


Spiral — X_test | Layer 1, Node 10

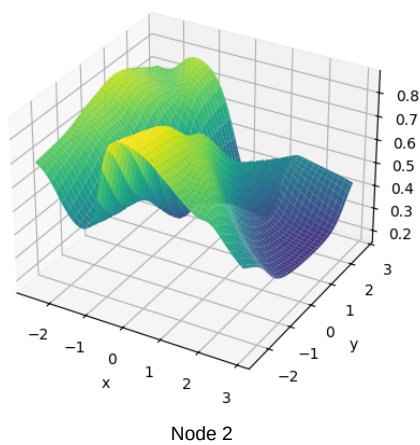


Spiral - Test - Hidden layer 2

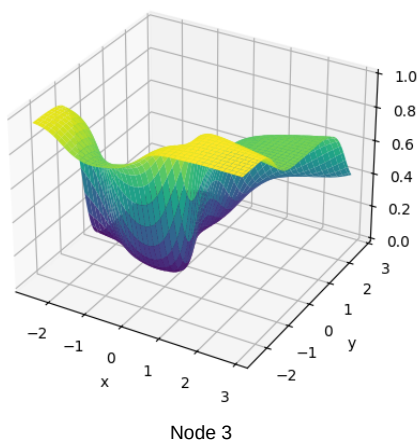
Spiral — X_test | Layer 2, Node 1



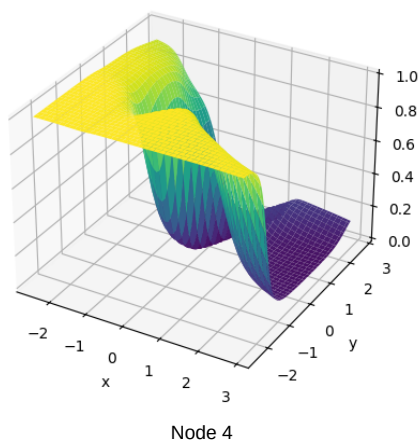
Spiral — X_test | Layer 2, Node 2



Spiral — X_test | Layer 2, Node 3

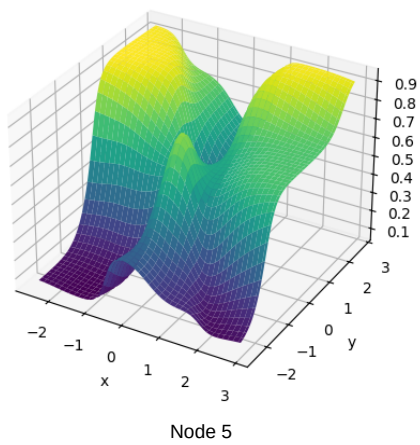


Spiral — X_test | Layer 2, Node 4

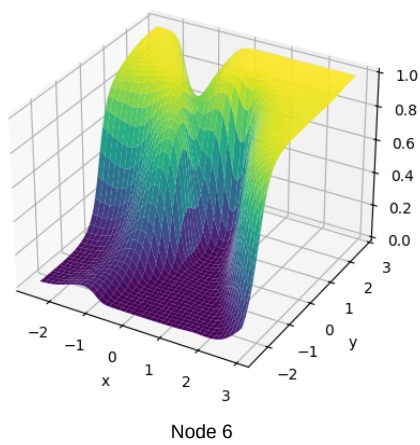


Spiral - Test - Hidden layer 2 (continued)

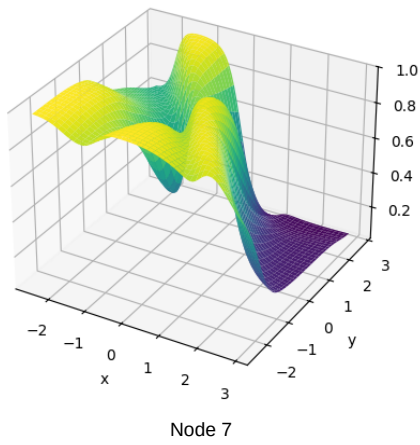
Spiral — X_test | Layer 2, Node 5



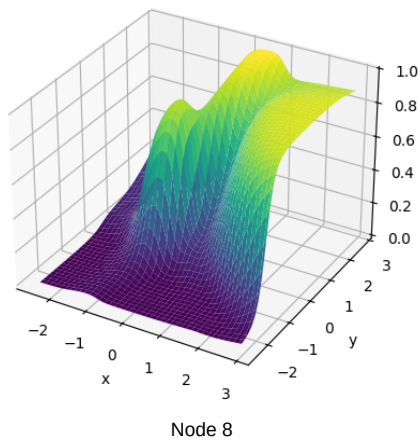
Spiral — X_test | Layer 2, Node 6



Spiral — X_test | Layer 2, Node 7

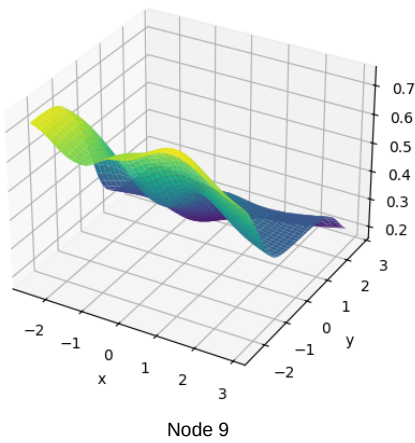


Spiral — X_test | Layer 2, Node 8

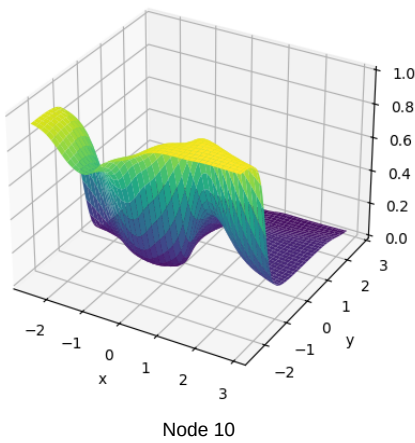


Spiral - Test - Hidden layer 2 (continued)

Spiral — X_test | Layer 2, Node 9

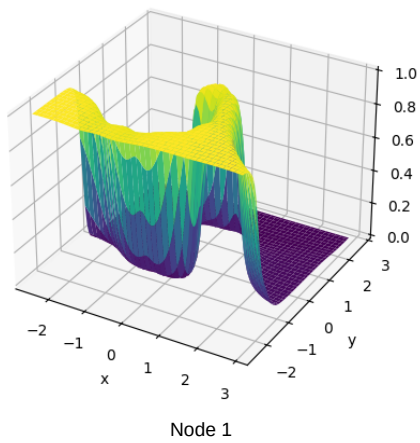


Spiral — X_test | Layer 2, Node 10

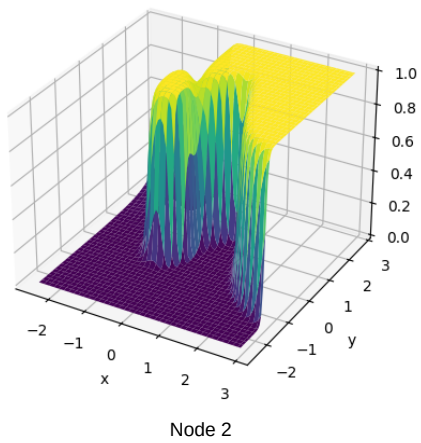


Spiral - Test - Output layer

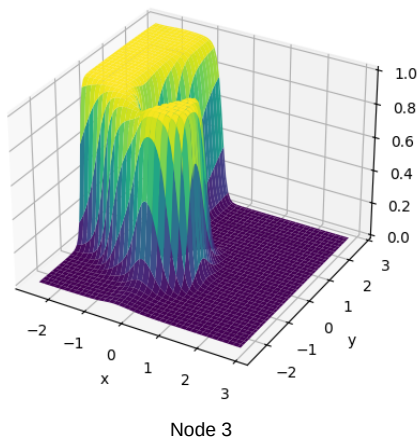
Spiral — X_test | Layer 3, Node 1



Spiral — X_test | Layer 3, Node 2



Spiral — X_test | Layer 3, Node 3



15. Comparison with the Single-Neuron Model

The notebook also trains a three-output single-neuron model with no hidden layer. The comparison uses the same two-dimensional input representation, sigmoid output and squared-error learning rule.

Dataset	Model	Training Accuracy	Test Accuracy
Gaussian	Best FCNN (2, 8, 3)	99.89%	100.00%
Gaussian	Single neuron	99.89%	99.67%
Spiral	Best FCNN (2, 10, 10, 3)	99.78%	100.00%
Spiral	Single neuron	51.89%	46.00%

The Gaussian dataset is simple enough that the single-neuron model performs almost as well as the FCNN. The spiral dataset shows the main benefit of hidden layers: the single-neuron model falls to 46.00% test accuracy, whereas the selected two-hidden-layer FCNN reaches 100.00% test accuracy.

16. Inferences and Discussion

Gaussian dataset. All tested FCNN architectures reached at least 99.67% validation accuracy, and four of the six configurations reached 100.00%. The selected (2, 8, 3) model is therefore already more than sufficient for this linearly separable dataset.

Spiral dataset. The smaller (2, 5, 5, 3) and (2, 8, 8, 3) models achieve 98.67% and 97.67% validation accuracy respectively. Increasing the hidden representation to (2, 10, 10, 3) raises validation accuracy to 100.00%. Several larger models also reach 100.00%, showing that additional capacity beyond this point does not improve the validation score for this run.

Generalization. The selected Gaussian and spiral FCNNs both achieve 100.00% validation and test accuracy. The confusion matrices for the selected models are diagonal, indicating that every example in those held-out sets was assigned to the correct class.

Representation learning. The node-response plots show how successive hidden layers transform the two-dimensional input into intermediate nonlinear representations. This is especially relevant to the spiral dataset, where a shallow linear model cannot reproduce the curved class boundaries.

17. Conclusion

The executed notebook satisfies the core implementation requirements of Task 1. Two synthetic three-class datasets were generated, split into training/validation/test subsets, and used to train fully connected neural networks with backpropagation and stochastic gradient descent from scratch.

The best Gaussian architecture selected by validation accuracy is (2, 8, 3), with 100.00% validation and 100.00% test accuracy. The best spiral architecture is (2, 10, 10, 3), also with 100.00% validation and 100.00% test accuracy. The single-neuron comparison further demonstrates that hidden layers are particularly important for nonlinear spiral classification.

18. Requirements Checklist

- Two synthetic three-class datasets implemented.
- 60% / 20% / 20% per-class train/validation/test split.
- One hidden layer for Dataset 1.
- Two hidden layers for Dataset 2.
- Different hidden-node counts tested.
- SGD with batch size 1.
- Squared error as instantaneous loss.
- Average error versus epochs for the selected architectures.
- Decision-region plots with training data.
- Validation confusion matrices for every tested architecture.
- Validation and test confusion matrices for the selected architectures.
- Training, validation and test node-output plots for hidden and output nodes.
- Comparison with a single-neuron model.
- Observations and conclusions based on the executed results.